

Modul 10: Javascript Lanjutan

Jevinson Imanuel Peuru

707012500101

D4SIKC-49-04

Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep dasar JavaScript sebagai bahasa pemrograman web.
2. Memahami fitur Modern JavaScript (ES6+) dan perannya sebagai fondasi React.
3. Mampu menggunakan sintaks ES6+ secara tepat dan konsisten.
4. Memahami konsep state, immutability, dan functional programming.
5. Mampu membangun aplikasi sederhana berbasis state menggunakan Vanilla JavaScript.

Alat dan Bahan

Untuk menjalankan modul ini, diperlukan komputer atau laptop dengan sistem operasi Windows, Linux, atau macOS, minimal prosesor 1 GHz, RAM 2GB (disarankan 4GB atau lebih), serta browser modern seperti Google Chrome, Mozilla Firefox, atau Microsoft Edge.

Perangkat lunak yang digunakan meliputi:

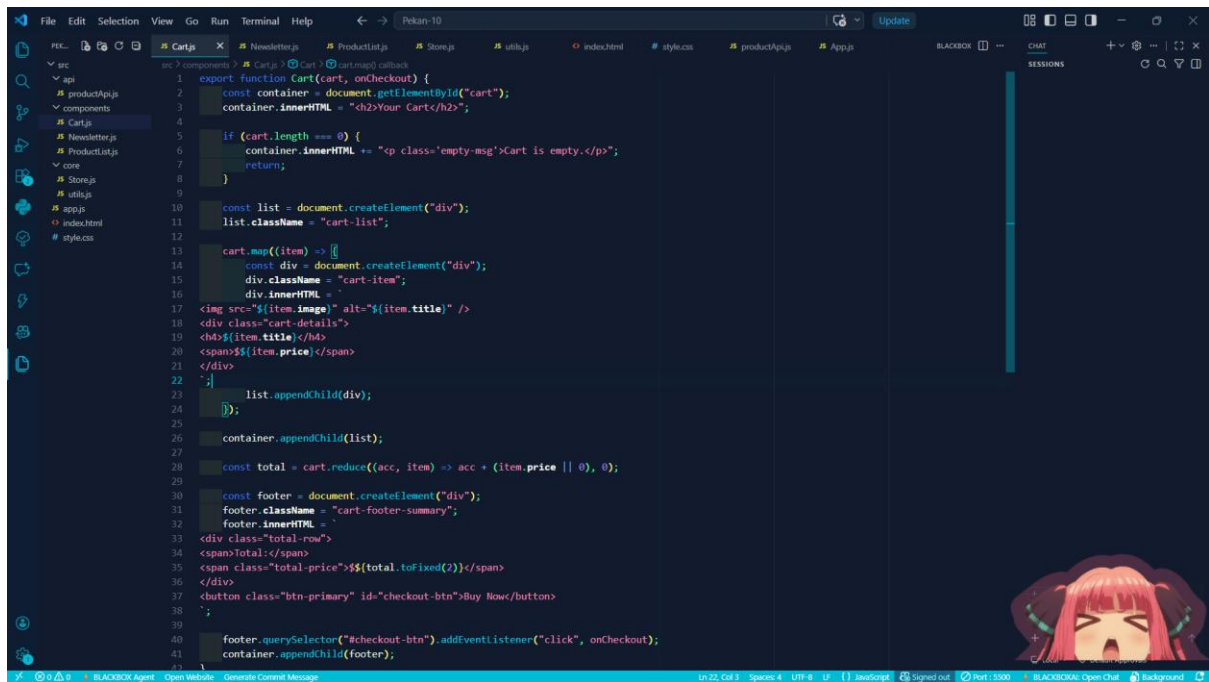
- Code editor (Visual Studio Code, Sublime Text, atau sejenisnya)
- Web browser

Bahan yang dibutuhkan:

- File HTML dasar
- File JavaScript (.js)

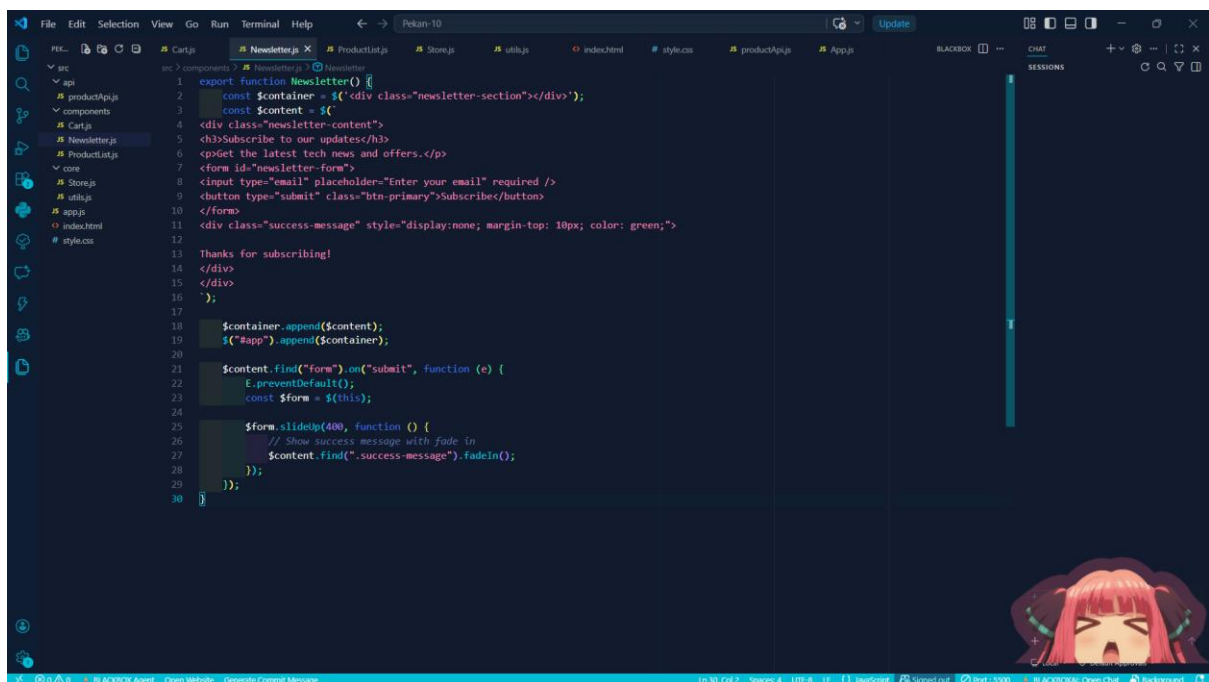
Praktikum

1. Cart.js



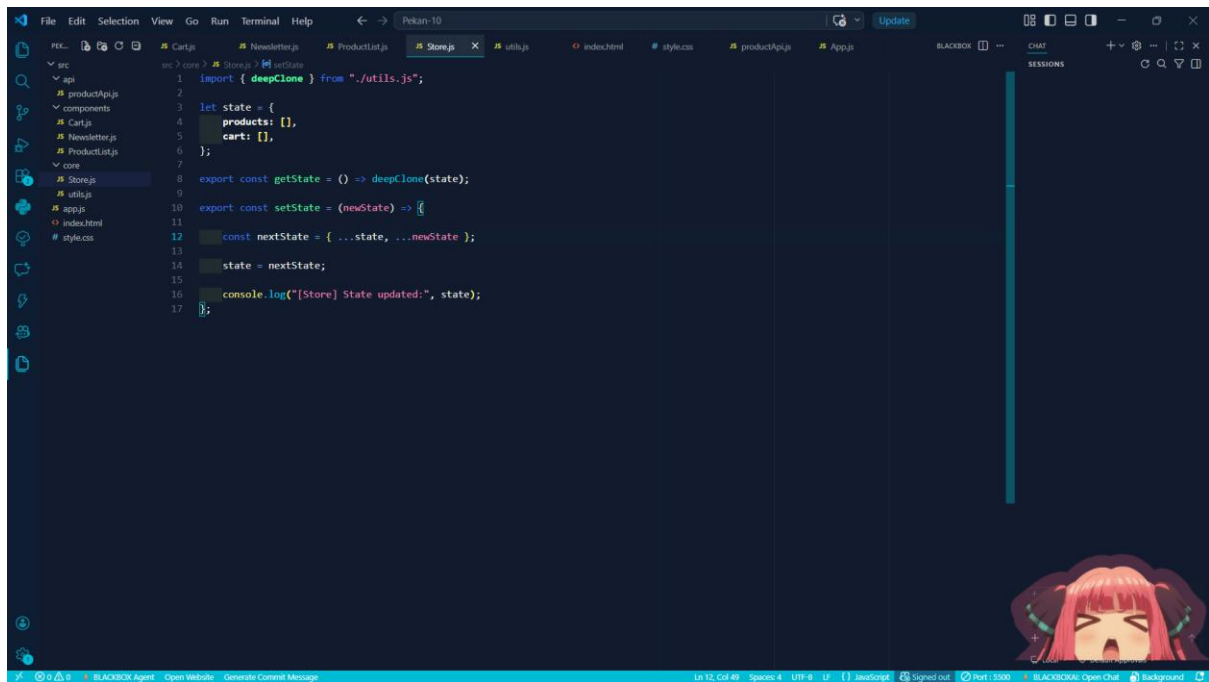
```
1 export function Cart(cart, onCheckout) {
2   const container = document.getElementById("cart");
3   container.innerHTML = "<h2>Your Cart</h2>";
4
5   if (cart.length === 0) {
6     container.innerHTML += "<p class='empty-msg'>Cart is empty.</p>";
7     return;
8   }
9
10  const list = document.createElement("div");
11  list.className = "cart-list";
12
13  cart.map((item) => {
14    const div = document.createElement("div");
15    div.className = "cart-item";
16    div.innerHTML = `
17      <img src='${item.image}' alt='${item.title}' />
18      <div class='cart-details'>
19        <h4>${item.title}</h4>
20        <span>${item.price}</span>
21      </div>
22    `;
23    list.appendChild(div);
24  });
25
26  container.appendChild(list);
27
28  const total = cart.reduce((acc, item) => acc + (item.price || 0), 0);
29
30  const footer = document.createElement("div");
31  footer.className = "cart-footer-summary";
32  footer.innerHTML = `
33    <div class='total-row'>
34      <span>Total</span>
35      <span class='total price'>${total.toFixed(2)}</span>
36    </div>
37    <button class='btn-primary' id='checkout-btn'>Buy Now</button>
38  `;
39
40  footer.querySelector("#checkout-btn").addEventListener("click", onCheckout);
41  container.appendChild(footer);
42}
```

2. Newsletter.js



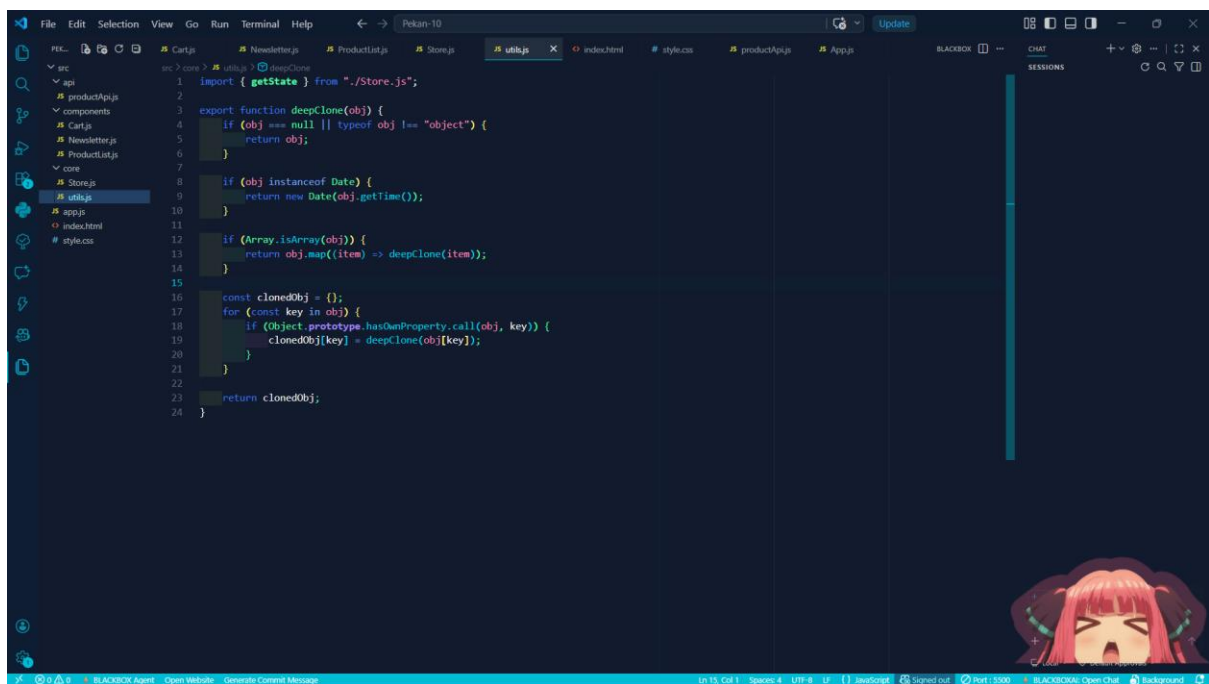
```
1 export function Newsletter() {
2   const $container = $('<div class='newsletter-section'></div>');
3   const $content = $(
4     <div class='newsletter-content'>
5       <h3>Subscribe to our updates</h3>
6       <p>Get the latest tech news and offers.</p>
7       <form id='newsletter-form'>
8         <input type='email' placeholder='Enter your email' required />
9         <button type='submit' class='btn-primary'>Subscribe</button>
10      </form>
11      <div class='success-message' style='display:none; margin-top: 10px; color: green;'>
12        Thanks for subscribing!
13      </div>
14    </div>
15  </div>
16  );
17
18  $container.append($content);
19  $('<#app>').append($container);
20
21  $content.find("<form>").on("<submit>", function (e) {
22    e.preventDefault();
23    const $form = $(this);
24
25    $form.slideUp(400, function () {
26      // Show success message with fade in
27      $content.find("<.success-message>").fadeIn();
28    });
29  });
30}
```

3. ProductList.js



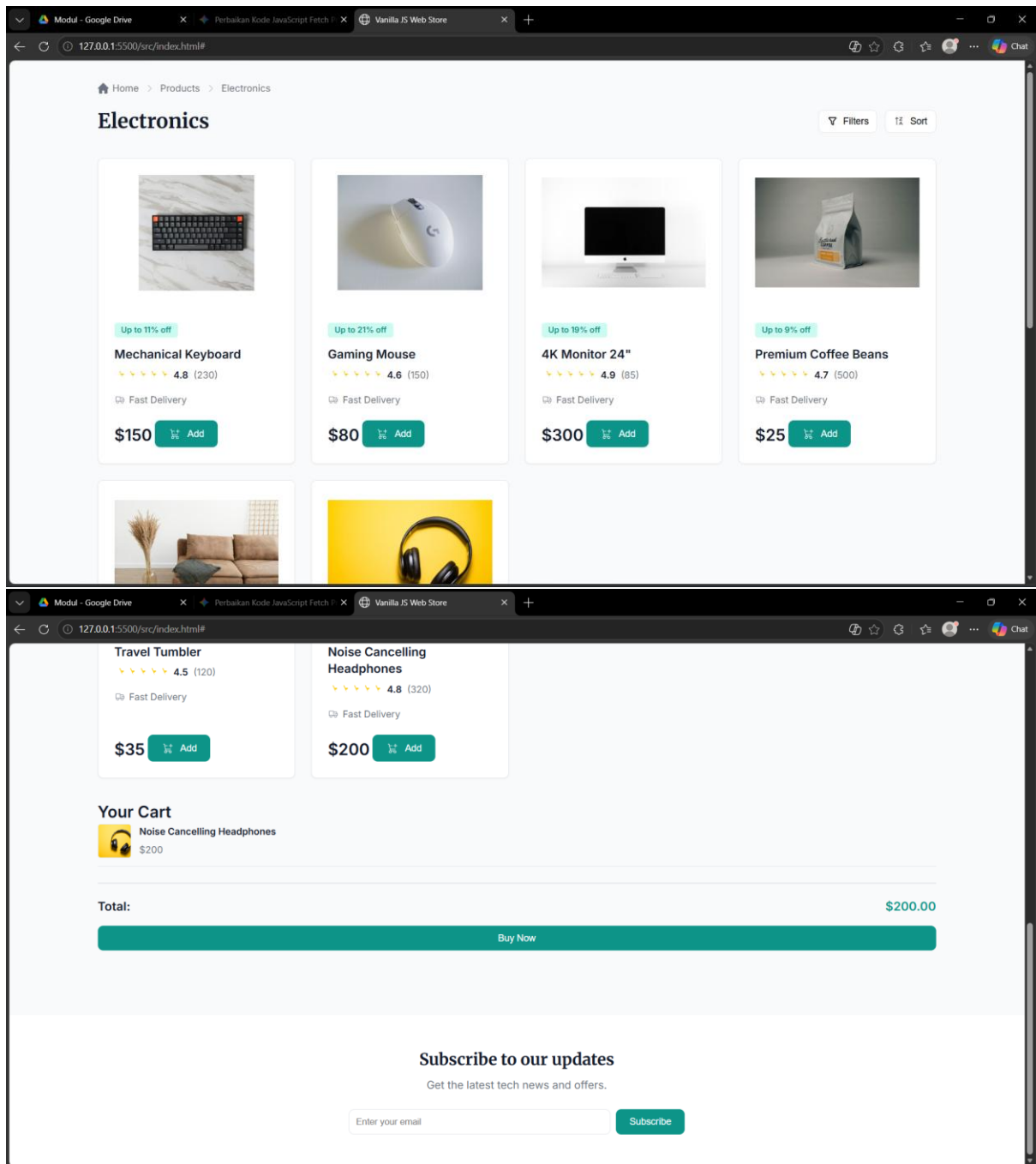
```
1 import { deepClone } from "../utils.js";
2
3 let state = {
4   products: [],
5   cart: [],
6 };
7
8 export const getState = () => deepClone(state);
9
10 export const setState = (newState) => {
11   const nextState = { ...state, ...newState };
12   state = nextState;
13   console.log("[Store] State updated:", state);
14 };
15
16
17
```

5. utils.js



```
1 import { getState } from "../Store.js";
2
3 export function deepClone(obj) {
4   if (obj === null || typeof obj !== "object") {
5     return obj;
6   }
7
8   if (obj instanceof Date) {
9     return new Date(obj.getTime());
10   }
11
12   if (Array.isArray(obj)) {
13     return obj.map(item => deepClone(item));
14   }
15
16   const clonedObj = {};
17   for (const key in obj) {
18     if (Object.prototype.hasOwnProperty.call(obj, key)) {
19       clonedObj[key] = deepClone(obj[key]);
20     }
21   }
22
23   return clonedObj;
24 }
```

6. Output



Kesimpulan

Berdasarkan hasil perancangan dan implementasi, dapat disimpulkan bahwa aplikasi *e-commerce* ini berhasil dibangun menggunakan *Vanilla JavaScript* dengan arsitektur *front-end* modern yang modular dan berbasis komponen, membuktikan efisiensi pengembangan sistem yang dinamis tanpa bergantung pada *framework* eksternal. Aplikasi ini mendemonstrasikan penguasaan manipulasi *Document Object Model* (DOM) yang presisi dan penerapan sistem *state management* kustom guna menyinkronkan data secara *real-time* antara *mock API* asinkron dengan antarmuka pengguna. Secara keseluruhan, integrasi fungsionalitas inti, seperti manajemen keranjang belanja dan tata letak visual

yang responsif, tidak hanya memenuhi standar teknis yang tinggi, tetapi juga menyajikan pengalaman pengguna (UX) yang intuitif, fungsional, dan profesional.

Link GitHub:

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/JevinsonImmanuelPeuru-707012500101>